

Introduction to Data Science and Engineering

- Basics of machine learning

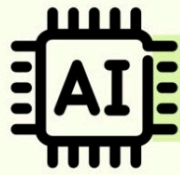


Some materials courtesy of
Rafael A. Irizarry, and are modified
from the original version.

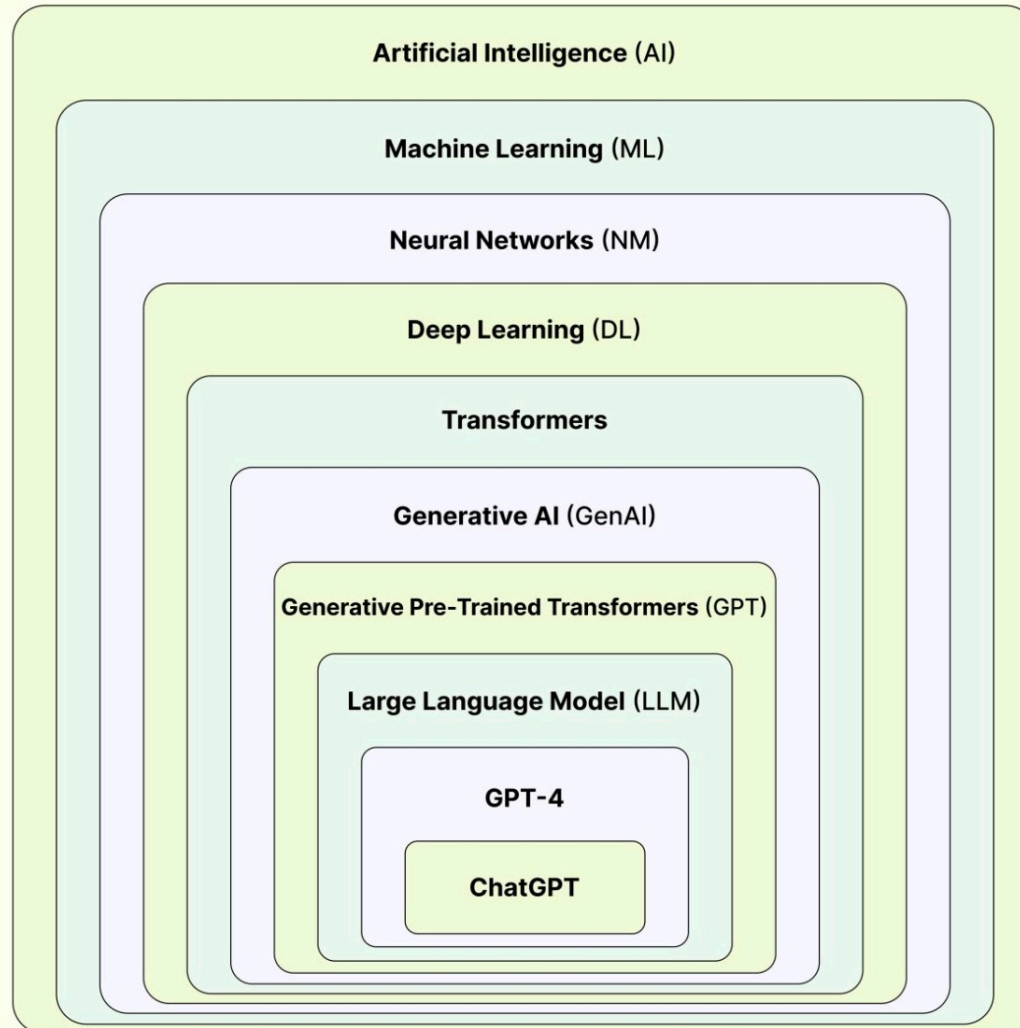
RB Luo / 羅銳邦

School of Computing and Data Science
University of Hong Kong

Some slides retrofitted by Michael Wu



AI Terminology



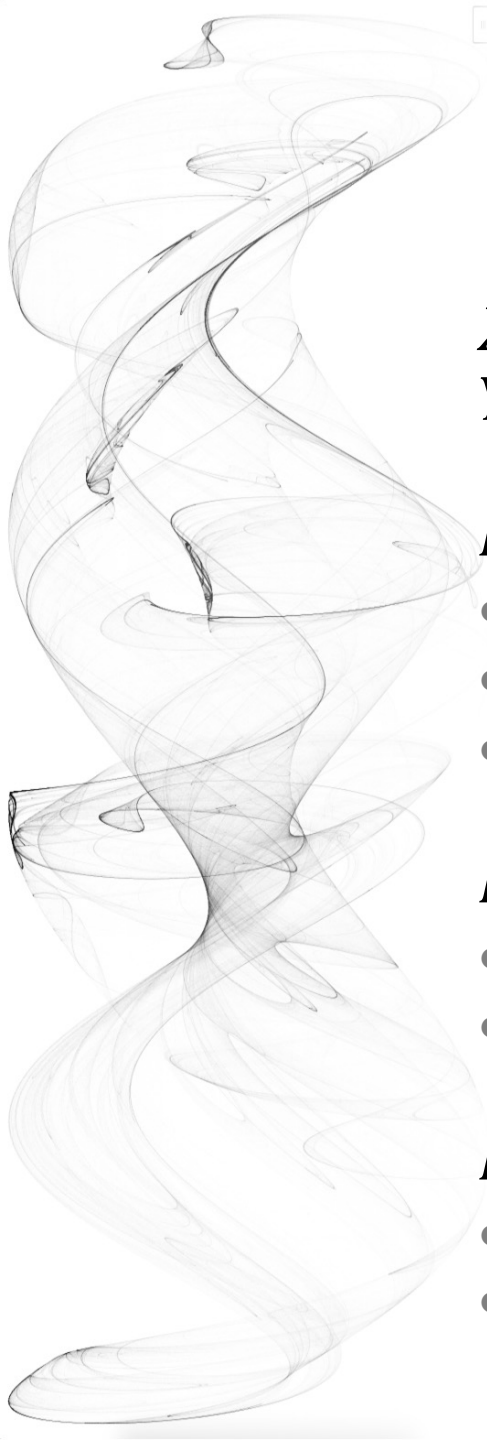
- AI and ML are often used interchangeably
- Don't split hairs with non-experts





In this lecture

- **Definition and notation**
- Evaluations
- Decision boundary
- What to use in practice



$P(X|Y), P(Y|X), P(X)$

X : observed data, images, texts, features, the cause, etc.

Y : underlying category, characteristics, outcome, the result, etc.

$P(Y|X)$: Prediction of labels given inputs. Discriminative modeling

- Given email content, is it a spam?
- Given lab results and medical images, diagnose the disease.
- Given a protein sequence, predict its structure

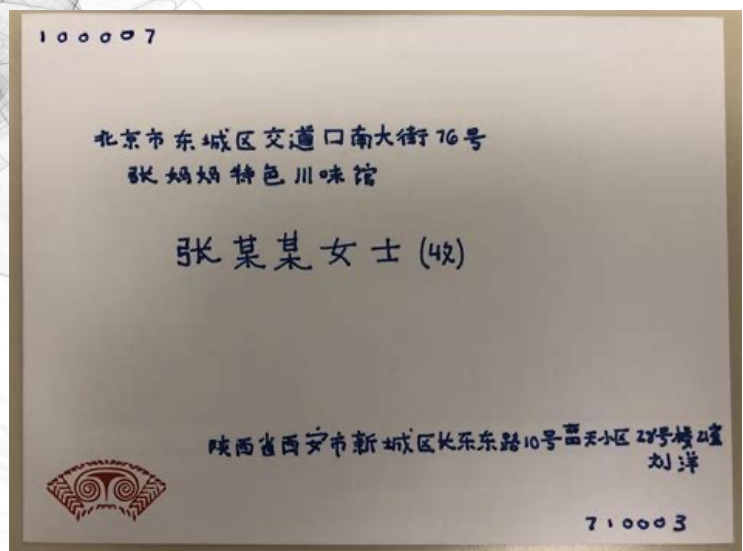
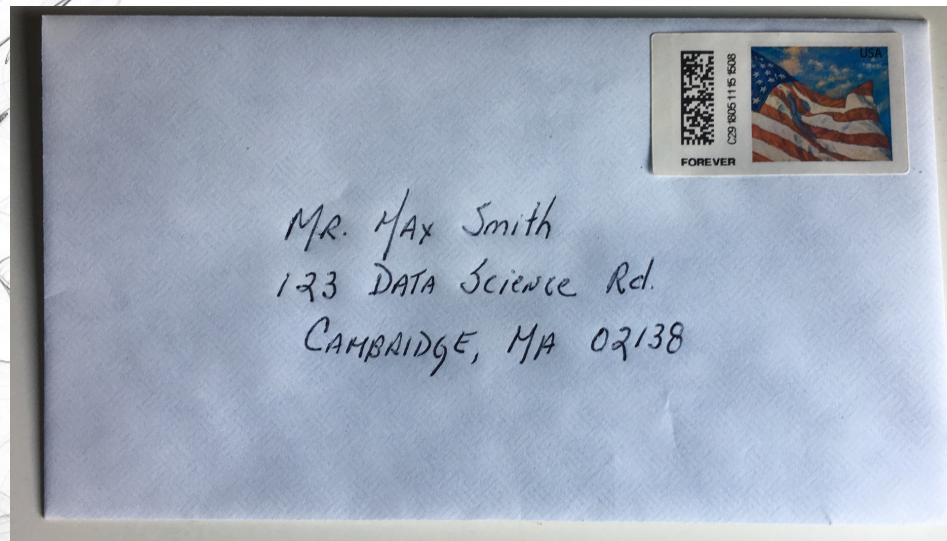
$P(X|Y)$: Modeling the data distribution given labels. Generative modeling

- Draw an image of a cat
- Generate a molecule that binds to a target protein

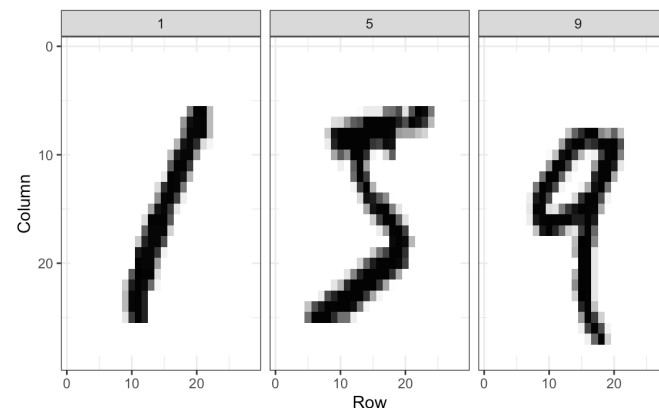
$P(X)$: Modeling the unconditional data distribution.

- Language modeling, next-word prediction
- Model normal heartbeats and identify irregular ones.

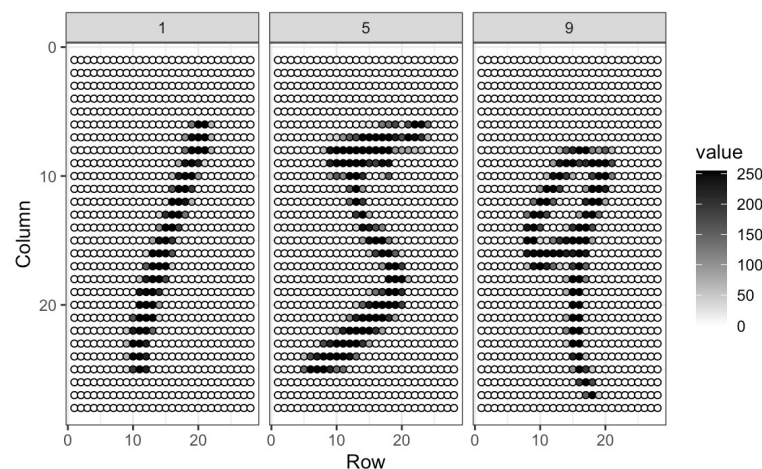
MNIST: Modified National Institute of Standards and Technology database



Capture



convert into 28x28 matrix



For each digitized image, we have a categorical label which can be one of 10 values (0,1,2,... , 9)



More Notation

- Y : the *outcome* we want to predict (the number)
 - When Y is a categorical variable (i.e., one out of K classes), this task is called classification
 - Binary classification is the special case when $K = 2$
 - When Y is a continuous variable, this task is called regression.
- X : the *features* we use to predict the outcome (the image)
- $P(Y|X)$: the machine model we will train. This model reads features (images) and predict outcomes (numbers)



More Notation

When the input has fixed number of features

- y_i : the *outcome* of the i -th sample.
- $x_{i,j}$: the j -th *feature* of the i -th sample. There are 784 values in a 28x28 matrix.
- $P(y_i|x_i) := f(x_{i,1}, x_{i,2}, x_{i,3}, \dots)$

outcome	feature 1	feature 2	feature 3	feature 4	feature 5
y_1	$x_{1,1}$	$x_{1,2}$	$x_{1,3}$	$x_{1,4}$	$x_{1,5}$
y_2	$x_{2,1}$	$x_{2,2}$	$x_{2,3}$	$x_{2,4}$	$x_{2,5}$
$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\vdots$
y_n	$x_{n,1}$	$x_{n,2}$	$x_{n,3}$	$x_{n,4}$	$x_{n,5}$

Working on the MNIST example

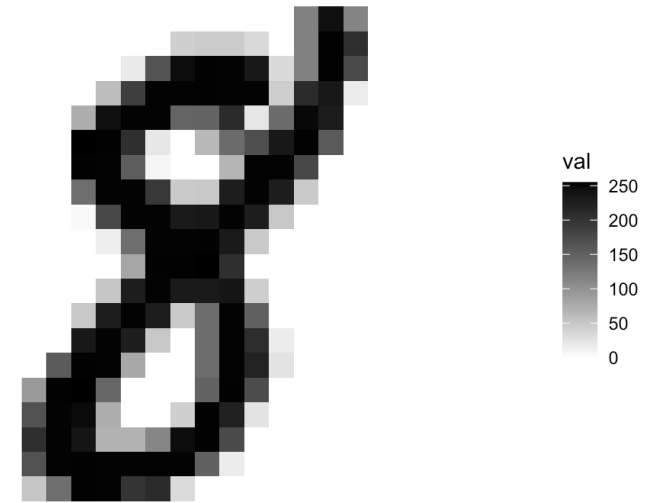
```
library(dslabs)
library(caret)
library(tidyverse)

mnist <- read_mnist()

# Visualize an example image
visualize_sample <- function(img, label, pred=NA){
  img_df <- expand_grid(x=seq(1:28), y=seq(1:28)) |>
    mutate(val = img)
  title <- paste("MNIST digit - label:", label)
  if (!is.na(pred)){
    title <- paste(title, "prediction:", pred)
  }
  img_df |>
    ggplot(aes(x, y, fill = val)) +
    geom_raster() +
    scale_fill_gradient(low = "white", high = "black") +
    scale_y_reverse() + # top-left = (1,1)
    coord_fixed() +
    theme_void() +
    labs(title = title)
}

idx <- 42
img <- mnist$train$images[idx,]
label <- mnist$train$labels[idx]
visualize_sample(img=img, label=label)
```

MNIST digit – label: 8



Working on the MNIST example

```
# Sample 1,000 training examples
set.seed(123) # For reproducibility
train_index <- sample(nrow(mnist$train$images), 1000)
x_train <- mnist$train$images[train_index, ] / 255.
y_train <- factor(mnist$train$labels[train_index])

# Sample 1,000 test examples
test_index <- sample(nrow(mnist$test$images), 1000)
x_test <- mnist$test$images[test_index, ] / 255.
y_test <- factor(mnist$test$labels[test_index])

# Add column names (required for caret)
colnames(x_train) <- paste0("pxl_", 1:ncol(x_train))
colnames(x_test) <- colnames(x_train)

# Create data frames
train_df <- data.frame(x_train, y = y_train)
test_df <- data.frame(x_test, y = y_test)
```

```
> train_df[1, 750:785]
      pxl_750 pxl_751 pxl_752 pxl_753 pxl_754 pxl_755 pxl_756
1           0           0           0           0           0           0           0
      pxl_757 pxl_758 pxl_759 pxl_760 pxl_761 pxl_762 pxl_763
1           0           0           0           0           0           0           0
      pxl_764 pxl_765 pxl_766 pxl_767 pxl_768 pxl_769 pxl_770
1           0           0           0           0           0           0           0
      pxl_771 pxl_772 pxl_773 pxl_774 pxl_775 pxl_776 pxl_777
1           0           0           0           0           0           0           0
      pxl_778 pxl_779 pxl_780 pxl_781 pxl_782 pxl_783 pxl_784 y
1           0           0           0           0           0           0           0 6
```

Working on the MNIST example

```
# Train two models: a KNN model and a XGBoost tree classifier
knn_grid <- expand.grid(k = 5)
knn_model <- train(
  y ~ .,
  method = "knn",
  data = train_df,
  tuneGrid = knn_grid,
  trControl = trainControl(method = "none"))

xgb_grid <- expand.grid(
  nrounds = 100, max_depth = 6, eta = 0.3, gamma = 0,
  colsample_bytree = 0.8, min_child_weight = 1, subsample = 0.8)
xgb_model <- train(
  y ~ .,
  method = "xgbTree",
  data = train_df,
  tuneGrid = xgb_grid,
  trControl = trainControl(method = "none"))
```

```
> xgb_model
eXtreme Gradient Boosting

1000 samples
784 predictor
10 classes: '0', '1', '2', '3', '4', '5', '6', '7', '8', '9'

No pre-processing
Resampling: None
```

```
> y_hat_knn <- predict(knn_model, train_df, type = "raw")
> print(mean((y_hat_knn == y_train)))
[1] 0.933
>
> y_hat_xgb <- predict(xgb_model, train_df, type = "raw")
> print(mean((y_hat_xgb == y_train)))
[1] 1
```

- How can we properly evaluate a model's performance?
- How can we tell which model is better?



In this lecture

- Definition and notation
- **Evaluations**
- Decision boundary
- What to use in practice



Evaluations

- Accuracy: $\frac{\text{Number of exact matches between predictions and labels}}{\text{Total count of samples}}$
- Imagine doing drug efficacy prediction: 1 positive case out of a total of 100 samples.
 - A model picks out 5 most promising candidates, including the real one. Accuracy is 0.96
 - A model predicts negative for everything. Accuracy is 0.99



Evaluation metrics

Before we start describing approaches to optimize the way we build algorithms, we first need to **define what we mean when we say one approach is better than another quantitatively**

For our first introduction to machine learning concepts, we will start with a very simple example: **how to predict gender using height**

We use the caret package (short for Classification And REgression Training), which is a pretty powerful machine learning library in R, after this example, we will see more details



Evaluation metrics

```
R 4.2.2 · ~/   
> library(tidyverse)  
> library(caret)  
> library(dslabs)  
> data(heights)
```

```
R 4.2.2 · ~/   
> y <- heights$sex  
> x <- heights$height
```

We start by defining the outcome and predictors. In this case, predictor is height, outcome is categorical and is either Male or Female. We know that we will not be able to predict Y very accurately based on X because male and female average heights are not that different relative to within group variability. But can we do better than guessing? To answer this question, we need a quantitative definition of better

Evaluation metrics - Training and test sets

Data for evaluating a model cannot be used for model training. Ultimately, a machine learning algorithm is evaluated on how it performs in the real world with completely new datasets. Practically, we split a dataset into two parts and act as if we don't know the outcome for one of these. We refer to the group for which we know the outcome, and use to develop the algorithm, as the training set. We refer to the group for which we pretend we don't know the outcome as the test set

```
R 4.2.2 · ~/ 
> set.seed(42)
> test_index <- createDataPartition(y, times = 1, p = 0.5, list = FALSE)
> test_set <- heights[test_index, ]
> train_set <- heights[-test_index, ]
```

times=1 means each index is included for sampling once, $p = 0.5$ means use 50% data for training, the remaining 50% for testing

We will now develop an algorithm using only the training set. Once we are done developing the algorithm, we will freeze it and evaluate it using the test set. The simplest way to evaluate the algorithm when the outcomes are categorical is by simply **reporting the proportion of cases that were correctly predicted in the test set**. This metric is usually referred to as **overall accuracy**

Evaluation metrics – Overall accuracy

To demonstrate the use of overall accuracy, we will build two competing algorithms and compare them

The first one: blind guessing

caret requires that categorical outcomes be coded as factors. So we also convert `y_hat` to factors using the `factor` function

```
R 4.2.2 · ~/    
> y_hat <- sample(c("Male", "Female"), length(test_index), replace = TRUE) |>  
+ factor(levels = levels(test_set$sex))
```

The overall accuracy is simply defined as the overall proportion that is predicted correctly

```
R 4.2.2 · ~/    
> mean(y_hat == test_set$sex)  
[1] 0.5161905
```

Not surprisingly, the accuracy is ~50%

Evaluation metrics – Overall accuracy

Can we do better?

Exploratory data analysis suggests we can because, on average, males are slightly taller than females

```
R 4.2.2 · ~/ 
> heights |> group_by(sex) |> summarize(mean(height), sd(height))
# A tibble: 2 × 3
  sex      `mean(height)` `sd(height)`
  <fct>         <dbl>         <dbl>
1 Female         64.9         3.76
2 Male           69.3         3.61
```

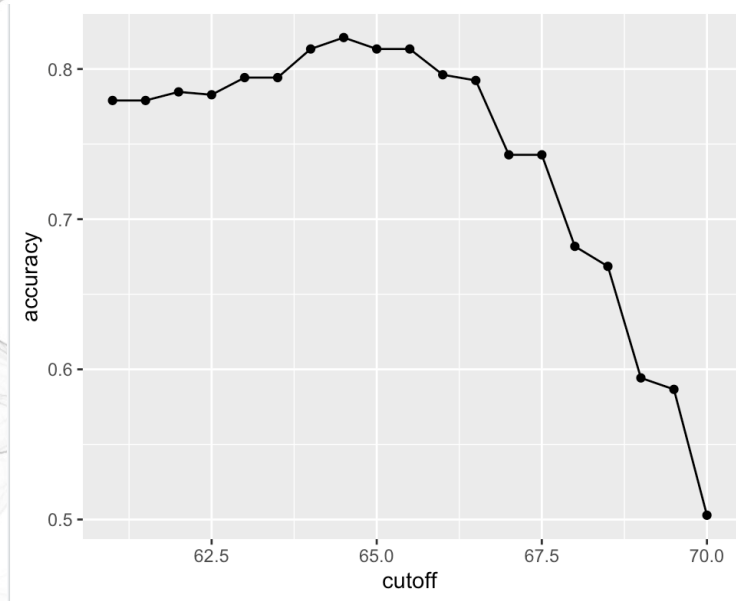
Let's try another simple approach:

Predict Male if height is within two standard deviations from the average male

```
R 4.2.2 · ~/ 
> y_hat <- ifelse(x > 62, "Male", "Female") |>
+ factor(levels = levels(test_set$sex))
> mean(y == y_hat)
[1] 0.7933333
```

The accuracy goes up from 0.50 to about 0.80, stupid method, but worked

Evaluation metrics – Overall accuracy



In the previous method, we can do better, is 62 the best cutoff? We can find the best cutoff in the training set (definitely not the test set) that maximizes overall accuracy

```
R 4.2.2 · ~/   
> cutoff <- seq(61, 70, 0.5)  
> accuracy <- map_dbl(cutoff, function(x){  
+   y_hat <- ifelse(train_set$height > x, "Male", "Female") |>  
+     factor(levels = levels(test_set$sex))  
+   mean(y_hat == train_set$sex)  
+ })
```

```
R 4.2.2 · ~/   
> best_cutoff <- cutoff[which.max(accuracy)]  
> best_cutoff  
[1] 64.5
```

The best cutoff is at 64.5, that achieved ~0.85 accuracy

```
R 4.2.2 · ~/   
> y_hat <- ifelse(test_set$height > best_cutoff, "Male", "Female") |>  
+ factor(levels = levels(test_set$sex))  
> mean(y_hat == test_set$sex)  
[1] 0.8419048
```

By testing on a dataset that we did not train on, we know our result is not due to cherry-picking a good result

Evaluation metrics – Confusion matrix

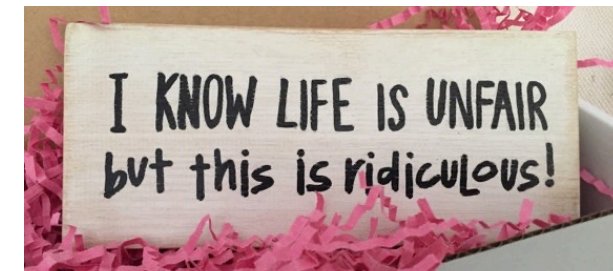
```
R 4.2.2 · ~/ ↵  
> table(predicted = y_hat, actual = test_set$sex)  
      actual  
predicted Female Male  
Female      58    22  
Male       61   384
```

```
R 4.2.2 · ~/ ↵  
> test_set |>  
+ mutate(y_hat = y_hat) |>  
+ group_by(sex) |>  
+ summarize(accuracy = mean(y_hat == sex))  
# A tibble: 2 × 2  
  sex      accuracy  
  <fct>      <dbl>  
1 Female    0.487  
2 Male     0.946
```

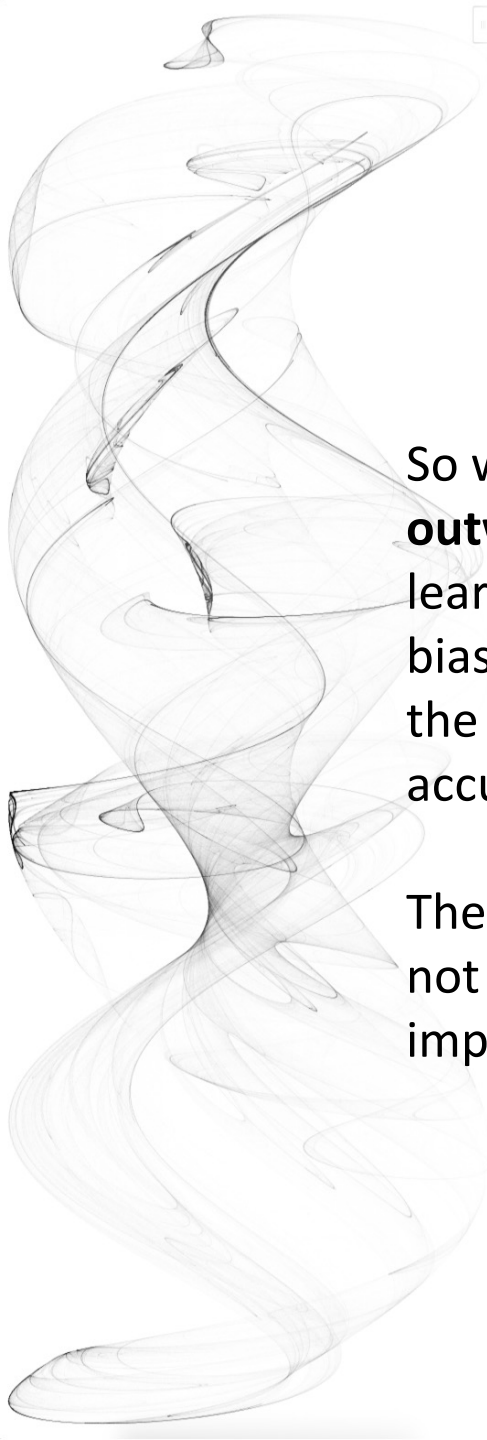
```
R 4.2.2 · ~/ ↵  
> mean(y == "Male")  
[1] 0.7733333
```

But using 64.5 as cutoff is not anywhere close to a reasonable method, albeit achieving 85% accuracy. Generally speaking, overall accuracy can be a deceptive measure, to see why, we use **confusion matrix**

If we study this table closely, it reveals a problem. If we compute the accuracy separately for each sex, we can see we are calling almost half of the females male



How can our overall accuracy be so high then? This is because the prevalence of males in this dataset is high (77.3%)



Evaluation metrics – Confusion matrix

So when computing overall accuracy, **the high percentage of mistakes made for females is outweighed by the gains in correct calls for men.** This can actually be a big problem in machine learning. If your training data is biased in some way, you are likely to develop algorithms that are biased as well. The fact that we used a test set does not matter because it is also derived from the original biased dataset. This is one of the reasons we look at metrics other than overall accuracy when evaluating a machine learning algorithm

There are several metrics that we can use to evaluate an algorithm in a way that prevalence does not cloud our assessment, and these can all be derived from the confusion matrix. A general improvement to using overall accuracy is to study **sensitivity** and **specificity** separately

We name the four entries of the confusion matrix

	Actually Positive	Actually Negative
Predicted positive	True positives (TP)	False positives (FP)
Predicted negative	False negatives (FN)	True negatives (TN)

Sensitivity is calculated

- "True positives over Actually Positive" – $TP / (TP + FN)$
- Intuition: How many Actually Positive were found
- aka Recall rate, or TPR (true positive rate)

Specificity is calculated as

- "True negatives over Actually Negative" – $TN / (TN + FP)$
- Intuition: How many Actually Negative were found
- aka TNR (true negative rate)

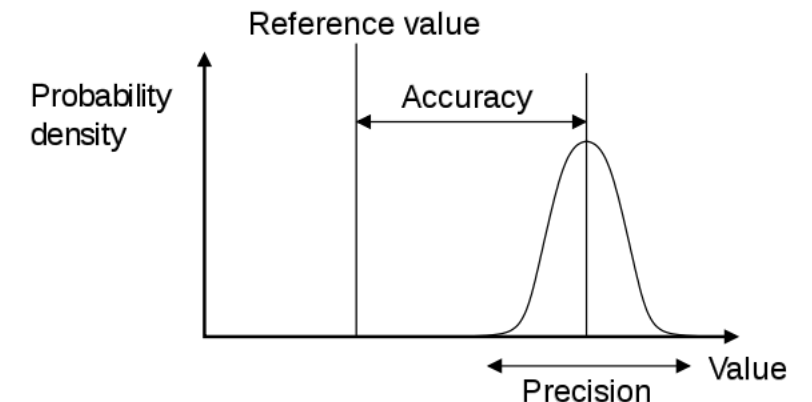
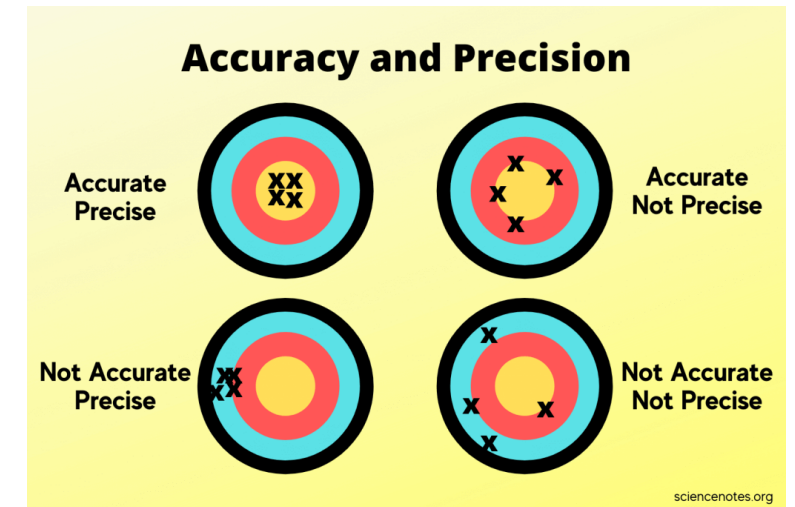
Precision is calculated as

- "True positives over Predicted Positive" – $TP / (TP + FP)$
- Intuition: How many Predicted Positive are actually positive
- aka PPV (positive predictive value)

Evaluation metrics

Accuracy is calculated

- "True over all total population" – $(TP + TN) / (P + N)$
- Intuition: Among all positives and negatives, how many of them are true?



Evaluation metrics

There are more evaluation metrics, for now, we only need accuracy, sensitivity, and specificity

Sources: [6][7][8][9][10][11][12][13][14] [view](#) · [talk](#) · [edit](#)

		Predicted condition			
Total population = P + N		Positive (PP)	Negative (PN)	Informedness, bookmaker informedness (BM) = TPR + TNR - 1	Prevalence threshold (PT) $= \frac{\sqrt{TPR \times FPR} - FPR}{TPR - FPR}$
Actual condition	Positive (P)	True positive (TP), hit	False negative (FN), type II error, miss, underestimation	True positive rate (TPR), recall, sensitivity (SEN), probability of detection, hit rate, power $= \frac{TP}{P} = 1 - FNR$	False negative rate (FNR), miss rate $= \frac{FN}{P} = 1 - TPR$
	Negative (N)	False positive (FP), type I error, false alarm, overestimation	True negative (TN), correct rejection	False positive rate (FPR), probability of false alarm, fall-out $= \frac{FP}{N} = 1 - TNR$	True negative rate (TNR), specificity (SPC), selectivity $= \frac{TN}{N} = 1 - FPR$
Prevalence $= \frac{P}{P + N}$	Positive predictive value (PPV), precision $= \frac{TP}{PP} = 1 - FDR$	False omission rate (FOR) $= \frac{FN}{PN} = 1 - NPV$	Positive likelihood ratio (LR+) $= \frac{TPR}{FPR}$	Negative likelihood ratio (LR-) $= \frac{FNR}{TNR}$	
Accuracy (ACC) $= \frac{TP + TN}{P + N}$	False discovery rate (FDR) $= \frac{FP}{PP} = 1 - PPV$	Negative predictive value (NPV) = $\frac{TN}{PN}$ $= 1 - FOR$	Markedness (MK), deltaP (Δp) $= PPV + NPV - 1$	Diagnostic odds ratio (DOR) $= \frac{LR+}{LR-}$	
Balanced accuracy (BA) $= \frac{TPR + TNR}{2}$	F_1 score $= \frac{2PPV \times TPR}{PPV + TPR} = \frac{2TP}{2TP + FP + FN}$	Fowlkes–Mallows index (FM) $= \sqrt{PPV \times TPR}$	Matthews correlation coefficient (MCC) $= \sqrt{TPR \times TNR \times PPV \times NPV}$ $- \sqrt{FNR \times FPR \times FOR \times FDR}$	Threat score (TS), critical success index (CSI), Jaccard index $= \frac{TP}{TP + FN + FP}$	

https://en.wikipedia.org/wiki/Precision_and_recall

Evaluation metrics

The caret function `confusionMatrix` computes all these metrics for us once we define what category “positive” is

```
R 4.2.2 · ~/ 
> cm <- confusionMatrix(data = y_hat, reference = test_set$sex)
> cm$overall
      Accuracy      Kappa AccuracyLower AccuracyUpper AccuracyNull
 8.419048e-01  4.899631e-01  8.078338e-01  8.720669e-01  7.733333e-01
AccuracyPValue McNemarPValue
 6.066659e-05  3.032101e-05
> cm$byClass
      Sensitivity      Specificity      Pos Pred Value
      0.4873950      0.9458128      0.7250000
Neg Pred Value      Precision      Recall
 0.8629213      0.7250000      0.4873950
      F1      Prevalence      Detection Rate
 0.5829146      0.2266667      0.1104762
Detection Prevalence      Balanced Accuracy
 0.1523810      0.7166039
```

We can see that the high overall accuracy is possible despite relatively low sensitivity. As we hinted at above, the reason this happens is because of the low prevalence (0.23): the proportion of females is low. Because prevalence is low, failing to predict actual females as females (low sensitivity) does not lower the accuracy as much as failing to predict actual males as males (low specificity). This is an example of why it is important to examine sensitivity and specificity and not just accuracy

Evaluation metrics – F1-score

One more thing, F1-score, which is commonly used in machine learning, aka f-measure, is the harmonic mean of precision and sensitivity, and is calculated as

$$F = \frac{2}{\frac{1}{Precision} + \frac{1}{Sensitivity}}$$

Alternatively

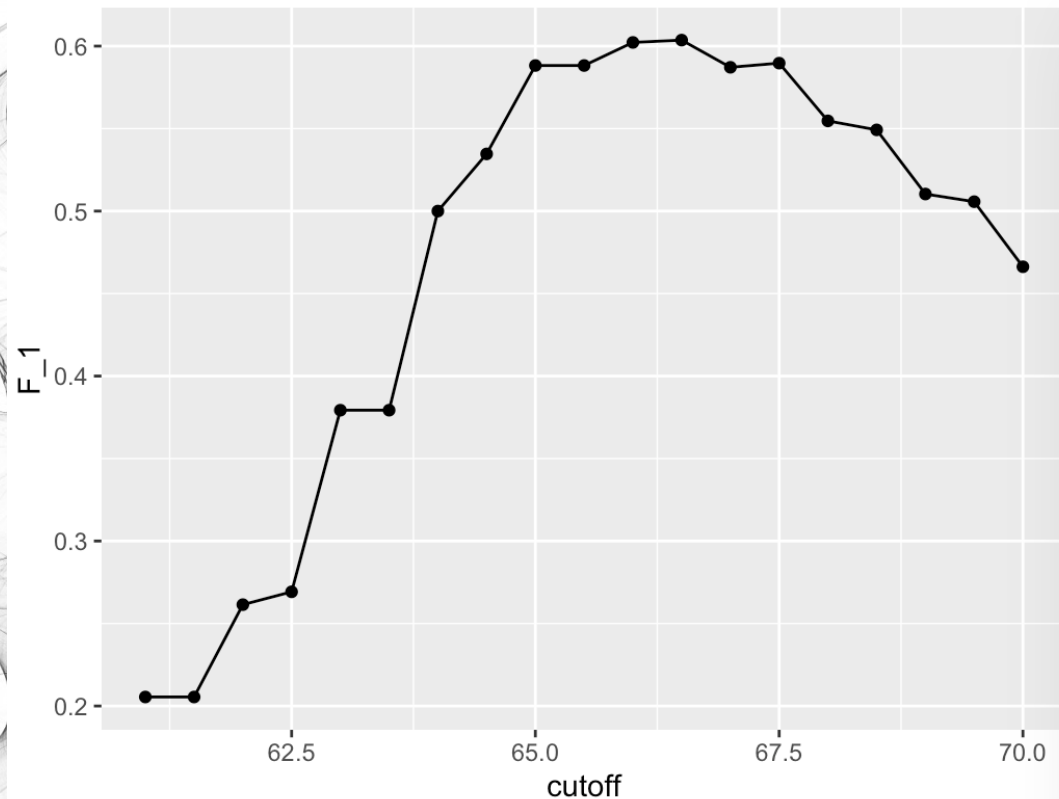
$$F = 2 \times \frac{Precision \times Sensitivity}{Precision + Sensitivity}$$

HG003 WGS 10x performance:

Model/ Caller	Testing Dataset	Overall Precision	Overall Recall	Overall F1	SNP Precision	SNP Recall	SNP F1
r941_prom_sup_g5014	Guppy5 sup 10x	96.01%	90.21%	93.02%	97.11%	96.53%	96.82%
r941_prom_sup_g506	Guppy5 sup 10x	97.38%	88.86%	92.93%	98.19%	95.49%	96.82%
PEPPER (r0.7)	Guppy5 sup 10x	96.39%	82.91%	89.14%	97.54%	89.24%	93.20%
Guppy3+4	Guppy4 hac 10x	88.96%	86.11%	87.51%	90.55%	93.35%	91.93%

An example of
how F1-score is
used

Evaluation metrics – F1-score

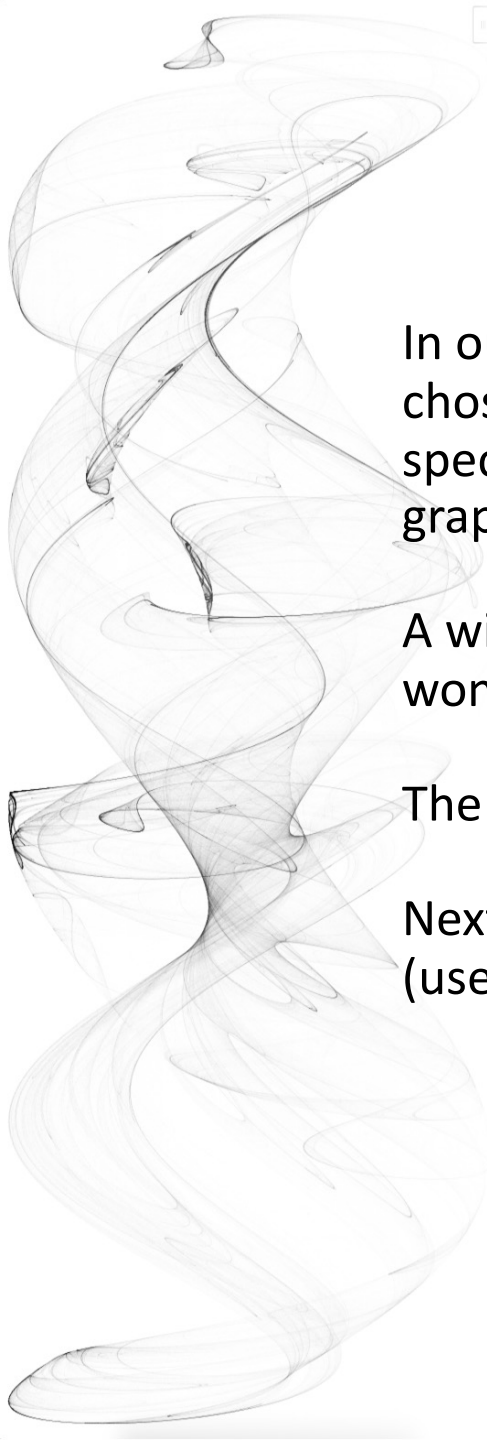


```
R 4.2.2 · ~/    
> y_hat <- ifelse(test_set$height > best_cutoff, "Male", "Female") |>  
+ factor(levels = levels(test_set$sex))  
> sensitivity(data = y_hat, reference = test_set$sex)  
[1] 0.7058824  
> specificity(data = y_hat, reference = test_set$sex)  
[1] 0.8349754
```

Let's retouch our prediction method 2, this time maximizing the F-score instead of overall accuracy. The `F_meas` function in the `caret` package that computes F1-score

```
R 4.2.2 · ~/    
> cutoff <- seq(61, 70, 0.5)  
> F_1 <- map_dbl(cutoff, function(x){  
+   y_hat <- ifelse(train_set$height > x, "Male", "Female") |>  
+   factor(levels = levels(test_set$sex))  
+   F_meas(data = y_hat, reference = factor(train_set$sex))  
+ })  
> max(F_1)  
[1] 0.6036364  
> best_cutoff <- cutoff[which.max(F_1)]  
> best_cutoff  
[1] 66.5
```

A cutoff of 66.5 makes more sense than 64.5. Furthermore, it balances the specificity and sensitivity of our confusion matrix

An abstract graphic on the left side of the slide, consisting of multiple overlapping, wavy, light gray lines that create a sense of movement and depth, resembling a stylized 'S' or a series of connected loops.

Evaluation metrics – ROC

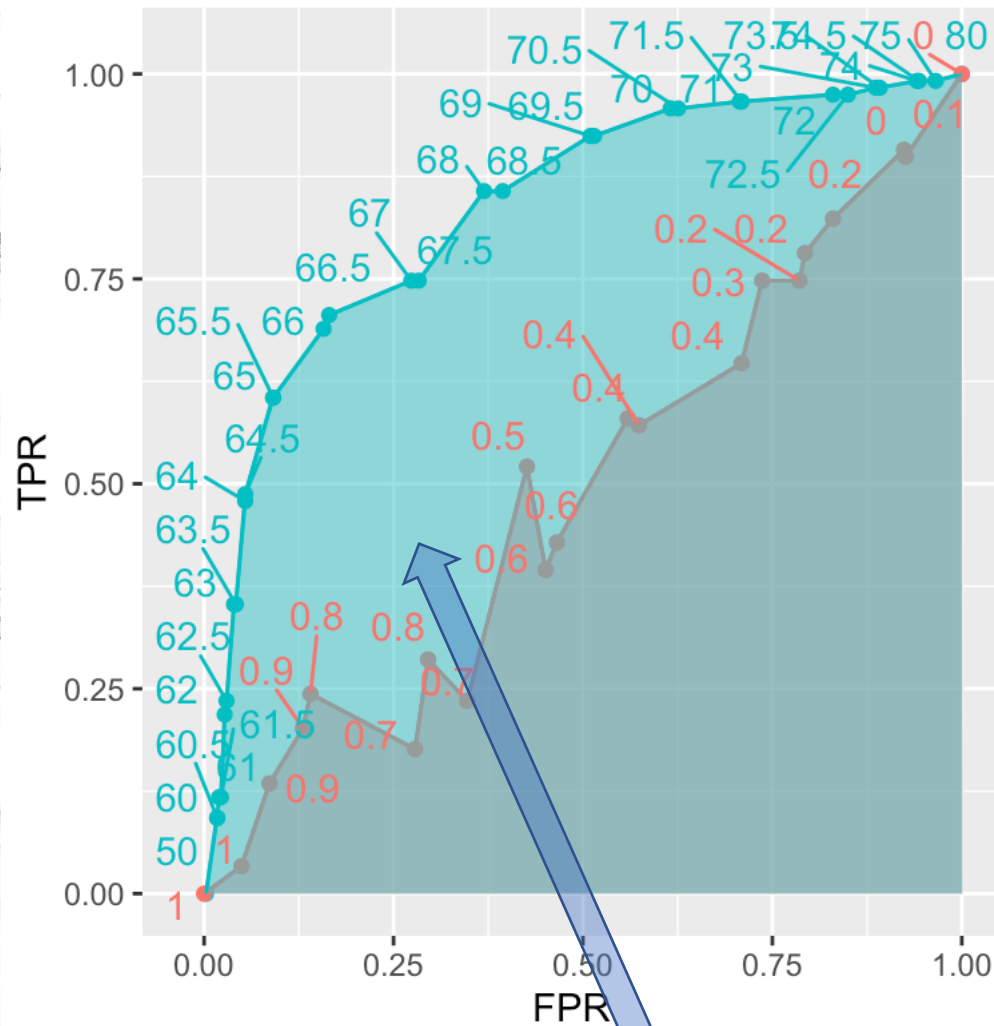
In our prediction method 2, firstly we have chosen cutoff 64.5 to maximize accuracy, and then chosen 66.5 to maximize F1-score. With different cutoffs we can get a different sensitivity and specificity. For this reason, a very common approach to evaluating methods is to compare them graphically by plotting both

A widely used plot that does this is the **receiver operating characteristic (ROC)** curve. If you are wondering where this name comes from, visit [Receiver operating characteristic – Wikipedia](#)

The ROC curve plots sensitivity (TPR) versus 1 - specificity or the false positive rate (FPR)

Next, we will use ROC curve to compare the performance of method 1 (blind guessing) and 2 (use height cutoff)

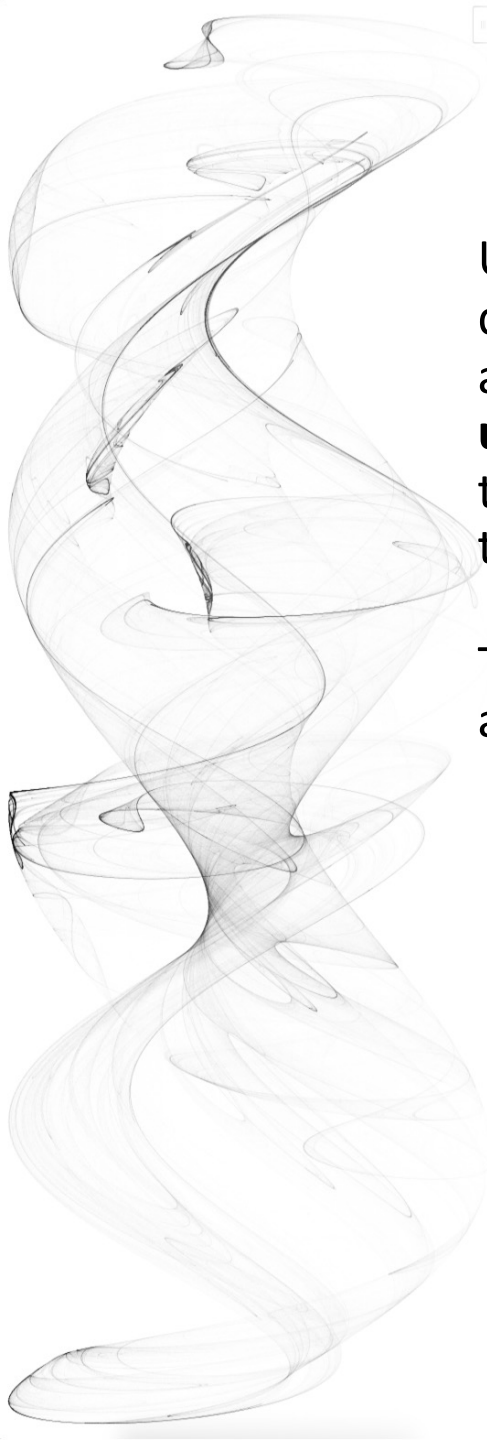
Evaluation metrics – ROC



The method with the largest AUC (Area Under Curve) in ROC wins

R 4.2.2 · ~/

```
> library(ggrepel)
> cutoffs <- c(50, seq(60, 75, 0.5), 80)
> tmp_1 <- map_df(cutoffs, function(x){
+   y_hat <- ifelse(test_set$height > x, "Male", "Female") |>
+     factor(levels = c("Female", "Male"))
+   list(method = "Height cutoff",
+         cutoff = x,
+         FPR = 1-specificity(y_hat, test_set$sex),
+         TPR = sensitivity(y_hat, test_set$sex))
+ })
> probs <- seq(0, 1, length.out = 21)
> tmp_2 <- map_df(probs, function(p){
+   y_hat <-
+     sample(c("Male", "Female"), n, replace = TRUE, prob=c(p, 1-p)) |>
+     factor(levels = c("Female", "Male"))
+   list(method = "Blind guessing",
+         cutoff = round(p,1),
+         FPR = 1 - specificity(y_hat, test_set$sex),
+         TPR = sensitivity(y_hat, test_set$sex))
+ })
There were 50 or more warnings (use warnings() to see the first 50)
> bind_rows(tmp_1, tmp_2) |>
+   ggplot(aes(FPR, TPR, label = cutoff, color = method)) +
+   geom_line() +
+   geom_point() +
+   geom_area(aes(fill=method),alpha = .5, position = 'identity') +
+   geom_text_repel(nudge_x = -0.03, nudge_y = 0.03,
+                   max.time = 5, max.overlaps = 30, show.legend = FALSE)
```



Evaluation metrics – loss function

Up to now we have described evaluation metrics that apply exclusively to categorical data. Specifically, for binary outcomes, we have described how sensitivity, specificity, accuracy, and F1-score can be used as quantification. **However, these metrics are not useful for continuous outcomes.** In this section, we describe how the general approach to defining “best” in machine learning is to define a loss function, which can be applied to both categorical and continuous data

The most commonly used loss function is the squared loss function. If $\hat{y}$ is our predictor and y is the observed outcome, the squared loss function is simply

$$(\hat{y} - y)^2$$

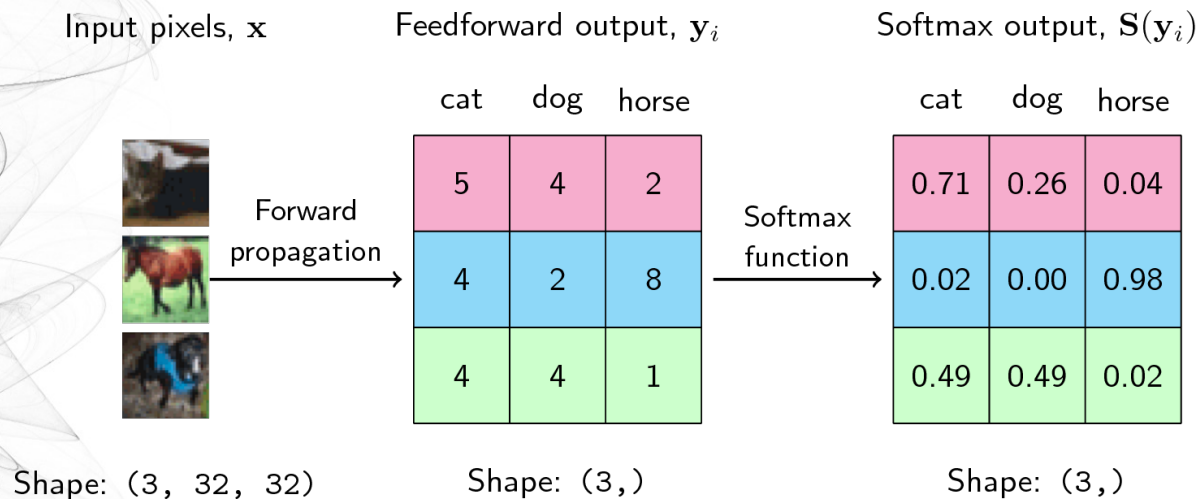
Because we often have a test set with many observations, say N , we use the mean squared error (MSE):

$$\text{MSE} = \frac{1}{N} \sum_{i=1}^N (\hat{y}_i - y_i)^2$$

Evaluation metrics – loss function

If the outcomes are binary, MSE is equivalent to one minus accuracy, since $(\hat{y} - y)^2$ is 0 if the prediction was correct and 1 otherwise. In general, **our goal is to build an algorithm that minimizes the loss so it is as close to 0 as possible**

In machine learning, the loss function for categorical data with more than two categories often uses softmax



<https://medium.com/@prabhuss73/softmax-function-why-exponentiation-matters-2c1e9ae01fdd>

Softmax function:

$$f_i(x) = \frac{\exp(x_i)}{\sum_j \exp(x_j)}$$

The softmax function is a function that turns a vector of K real values into a vector of K real values that sum to 1. The input values can be positive, negative, zero, or greater than one, but the softmax transforms them into values between 0 and 1, so that they can be interpreted as probabilities. If one of the inputs is small or negative, the softmax turns it into a small probability, and if an input is large, then it turns it into a large probability, but it will always remain between 0 and 1

[Softmax Function Definition | DeepAI](#)



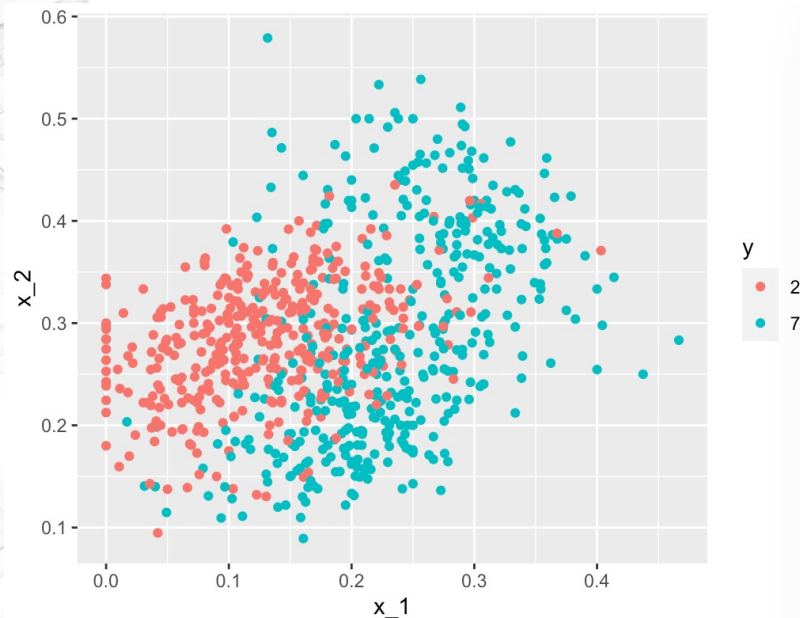
In this lecture

- Definition and notation
- Evaluations
- **Decision boundary**
- What to use in practice

Case study: is it a 2 or 7?

In the height/gender example above, we only had one feature. We actually do not consider these machine learning challenges, which are characterized by cases with many features. Let's go back to the digits example in which we had 784 features. For illustrative purposes, we will start by simplifying this problem to one with two features and two classes

Specifically, we define the challenge as building an algorithm that can determine if a digit is a 2 or 7 from the features. We are not quite ready to build algorithms with 784 predictors, so we will extract two simple predictors from the 784: the proportion of dark pixels that are in the upper left quadrant (X_1) and the lower right quadrant (X_2)



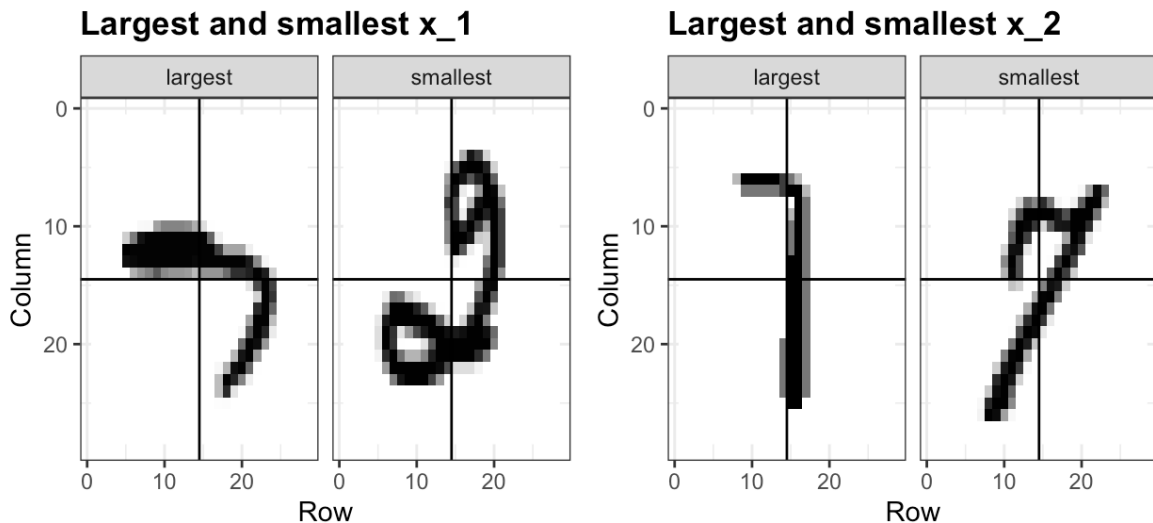
We then select a random sample of 1,000 digits, 800 in the training set and 200 in the test set. This dataset is available from the dslabs package. We can explore the data by plotting the two features and using colors to denote the labels

```
R 4.2.2 · ~/    
> library(tidyverse)  
> library(dslabs)  
> data("mnist_27")  
> mnist_27$train |> ggplot(aes(x_1, x_2, color = y)) + geom_point()
```

Case study: is it a 2 or 7?



We can immediately see some patterns. For example, if x_1 is very large, then the digit is probably a 7. Also, for smaller values of x_1 , the 2s appear to be in the mid range values of x_2 .



To illustrate how to interpret x_1 and x_2 , we include four example images. On the left are the original images of the two digits with the largest and smallest values for x_1 and on the right we have the images corresponding to the largest and smallest values of x_2 .

We can start getting a sense for why these predictors are useful, but also why the problem will be somewhat challenging.

Case study: is it a 2 or 7?

```
R 4.2.2 · ~/ ↵  
> fit <- mnist_27$train |>  
+ mutate(y = ifelse(y==7, 1, 0)) |>  
+ lm(y ~ x_1 + x_2, data = _)
```

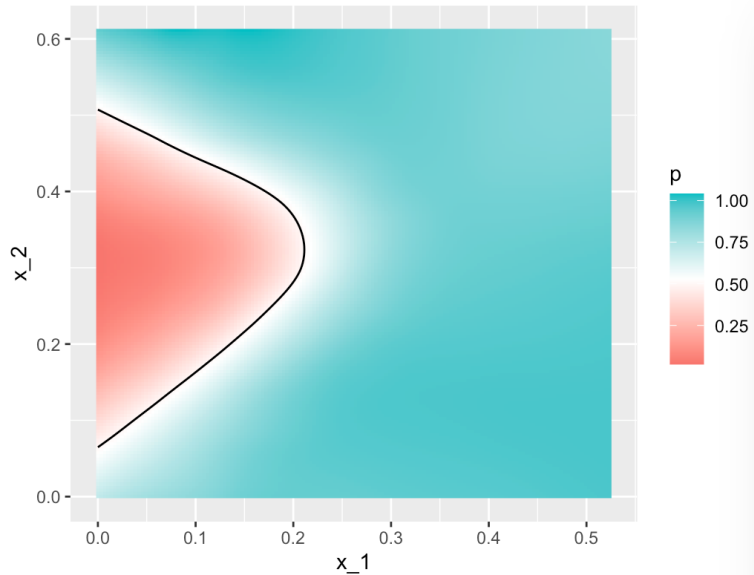
We haven't really learned any algorithms yet, so let's try building an algorithm using regression

```
R 4.2.2 · ~/ ↵  
> library(caret)  
> p_hat <- predict(fit, newdata = mnist_27$test)  
> y_hat <- factor(ifelse(p_hat > 0.5, 7, 2))  
> confusionMatrix(y_hat, mnist_27$test$y)$overall[["Accuracy"]]  
[1] 0.75
```

We can now build a decision rule based on the regression model we've just built

We get an accuracy well above a blind guess. Not bad for our first try. But can we do better?

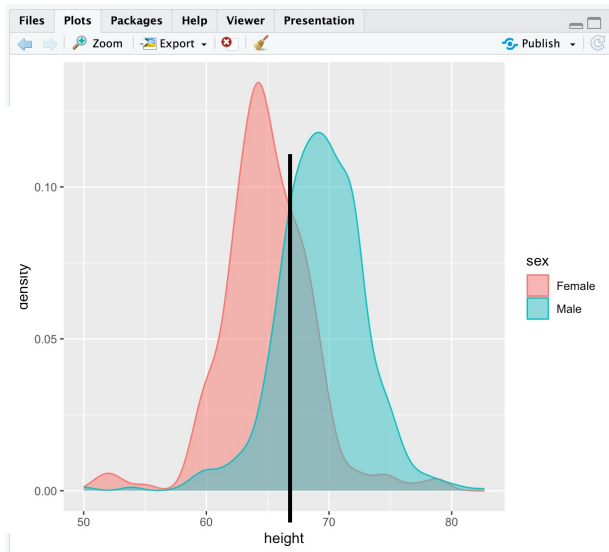
Case study: is it a 2 or 7?



We get an accuracy well above a blind guess. Not bad for our first try. But can we do better? To learn the limitation of an algorithm, let's see the theoretical best probability p of digit 7 against both features computed using all samples in the MNIST_27 dataset.

```
R 4.2.2 · ~/ 
> mnist_27$true_p |> ggplot(aes(x_1, x_2, z = p, fill = p)) +
  geom_raster() +
  scale_fill_gradientn(colors=c("#F8766D", "white", "#00BFC4")) +
  stat_contour(breaks=c(0.5), color="black")
```

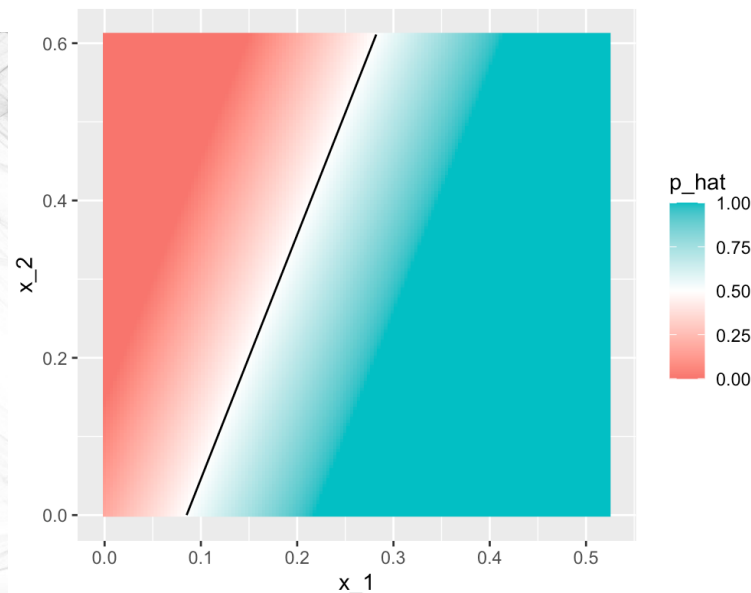
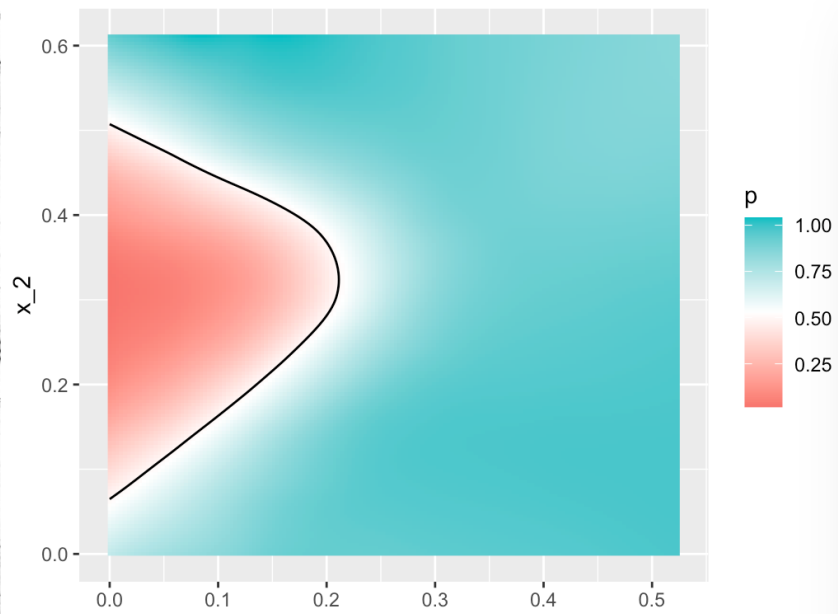
Given the height dataset for training, how good a perfect model can predict the gender correctly at 67-inch?



Theoretical best probability p of an input can be understood as, given a perfect model (classifier), what's the chance the model can make a correct prediction about the input

To learn the details of how the theoretical best probability p is generated, read Chapter 27.6

Case study: is it a 2 or 7?

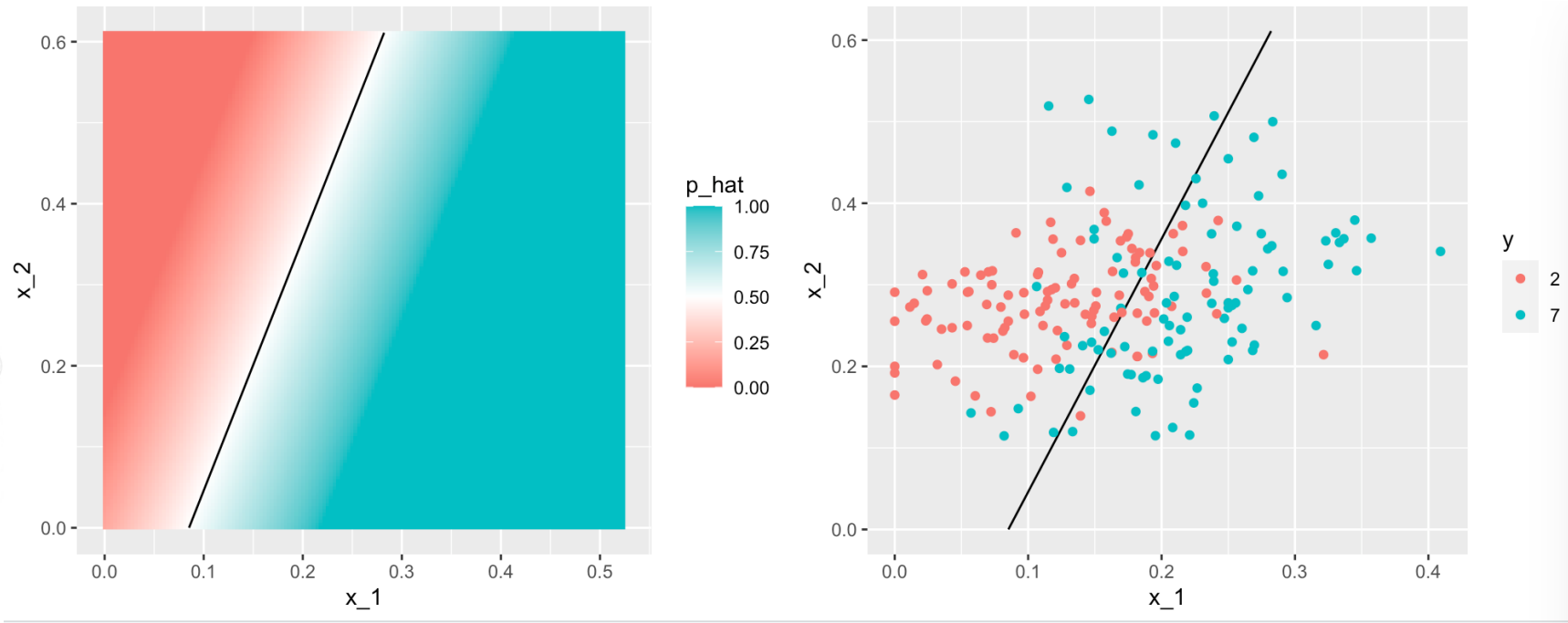


To start understanding the limitations of regression here, first note that with regression, the boundary to distinguish any two categories has to be a plane

Specifically, we used linear regression, i.e., x_2 is a linear function of x_1 , this implies that our regression approach has no chance of capturing the non-linear nature of the true p

On the left is a visual representation of the predicted probability $\hat{p}$ with the linear regression model. We used the squish function from the scales package to constrain estimates to be between 0 and 1. We can see where the mistakes were made by also showing the data and the boundary. They mainly come from low values x_1 that have either high or low value of x_2 . Linear regression can't catch this

Case study: is it a 2 or 7?



```
R 4.2.2 · ~/ 
> p_hat <- predict(fit, newdata = mnist_27$true_p[,c(1,2)])
p_hat <- scales::squish(p_hat, c(0, 1))
p1 <- mnist_27$true_p |> mutate(p_hat = p_hat) |>
  ggplot(aes(x_1, x_2, z=p_hat, fill=p_hat)) +
  geom_raster() +
  scale_fill_gradientn(colors=c("#F8766D", "white", "#00BFC4")) +
  stat_contour(breaks=c(0.5), color="black")
p2 <- mnist_27$true_p |> mutate(p_hat = p_hat) |>
  ggplot() +
  stat_contour(aes(x_1, x_2, z=p_hat), breaks=c(0.5), color="black") +
  geom_point(mapping = aes(x_1, x_2, color=y), data = mnist_27$test)
gridExtra::grid.arrange(p1, p2, ncol = 2)
```

We need something more flexible: a method that permits estimates with shapes other than a plane



The caret package

We have already learned regression as a machine learning algorithm. We will use several others in the following examples. To get a deep and broad coverage of machine learning algorithms in the future, you might be interested in courses including COMP3314 Machine Learning and COMP3340 Applied Deep Learning in the future

The caret package includes 238 different algorithms, summarized at <https://topepo.github.io/caret/available-models.html>. Keep in mind that caret does not include the needed packages and, to implement a package through caret, you still need to install the library. The required packages for each method are described in the package manual

The models below are available in `train`. The code behind these protocols can be obtained using the function `getModelInfo` or by going to the [github repository](#).

Show entries

Search:

Model ☐	<i>method</i> Value ☐	Type ☐	Libraries ☐	Tuning Parameters ☐
AdaBoost Classification Trees	adaboost	Classification	fastAdaboost	nIter, method
AdaBoost.M1	AdaBoost.M1	Classification	adabag, pLyr	mfinal, maxdepth,

The caret package – train and predict

The caret package provides functions to train a model using any of its algorithms, and to test a trained model, let's see some examples

R 4.2.2 · ~/

```
> library(caret)
> train_glm <- train(y ~ ., method = "glm", data = mnist_27$train)
> train_knn <- train(y ~ ., method = "knn", data = mnist_27$train)
```

Generalized Linear Model	glm	Classification, Regression	None
k-Nearest Neighbors	knn	Classification, Regression	k

R 4.2.2 · ~/

```
> y_hat_glm <- predict(train_glm, mnist_27$test, type = "raw")
> y_hat_knn <- predict(train_knn, mnist_27$test, type = "raw")
```

R 4.2.2 · ~/

```
> confusionMatrix(y_hat_glm, mnist_27$test$y)$overall[["Accuracy"]]
[1] 0.75
> confusionMatrix(y_hat_knn, mnist_27$test$y)$overall[["Accuracy"]]
[1] 0.84
```

The caret train function lets us train different algorithms using similar syntax

To make predictions, we can apply predict to the output of train directly

This permits us to quickly compare the algorithms. For example, we can compare the accuracy like this

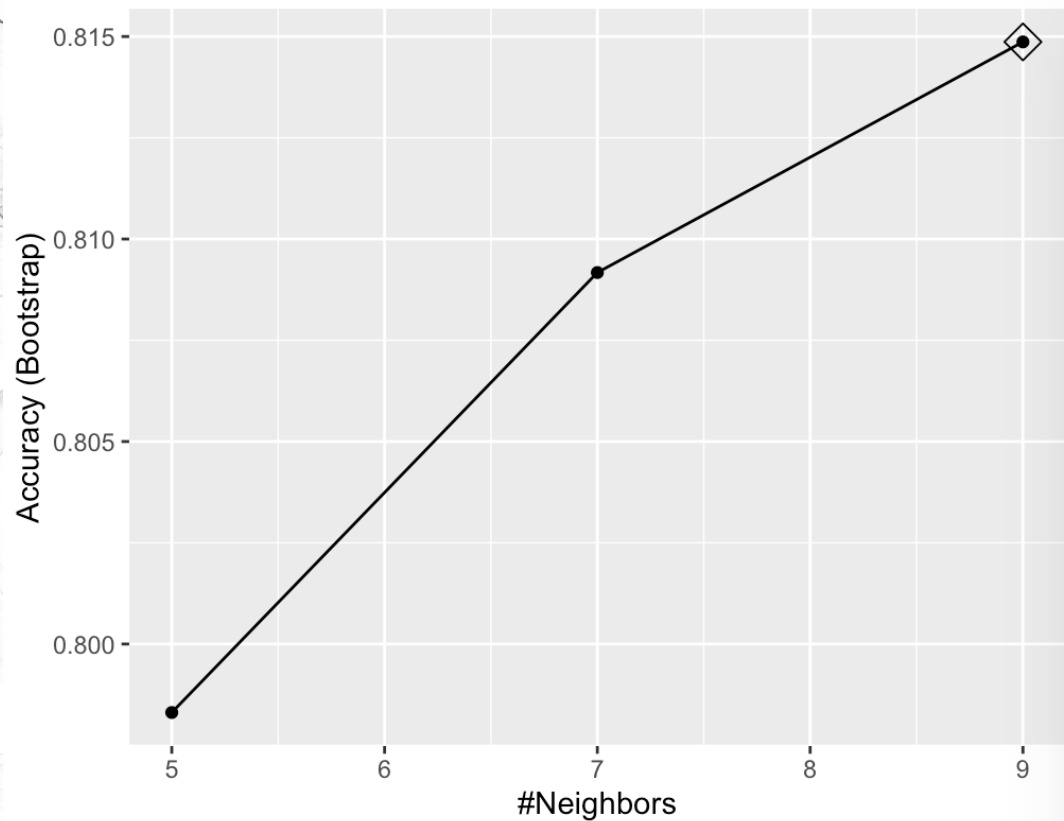
The caret package – parameter tuning

R 4.2.2 · ~/

```
> modelLookup("knn")
```

	model	parameter	label	forReg	forClass	probModel
1	knn	k #Neighbors		TRUE	TRUE	TRUE

When an algorithm includes a tuning parameter, train automatically uses cross validation to decide among a few default values, use the manual, or use for example ``modelLookup("knn")``

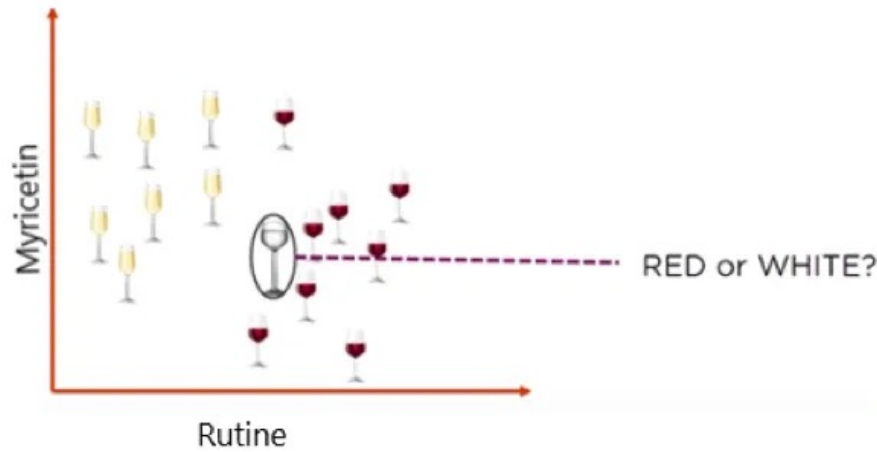


If we train a model with default values, we can quickly see the results of the cross validation using the ggplot function. The argument `highlight` highlights the top performant

R 4.2.2 · ~/

```
> set.seed(42)
> train_knn <- train(y ~ ., method = "knn", data = mnist_27$train)
> ggplot(train_knn, highlight = TRUE)
```

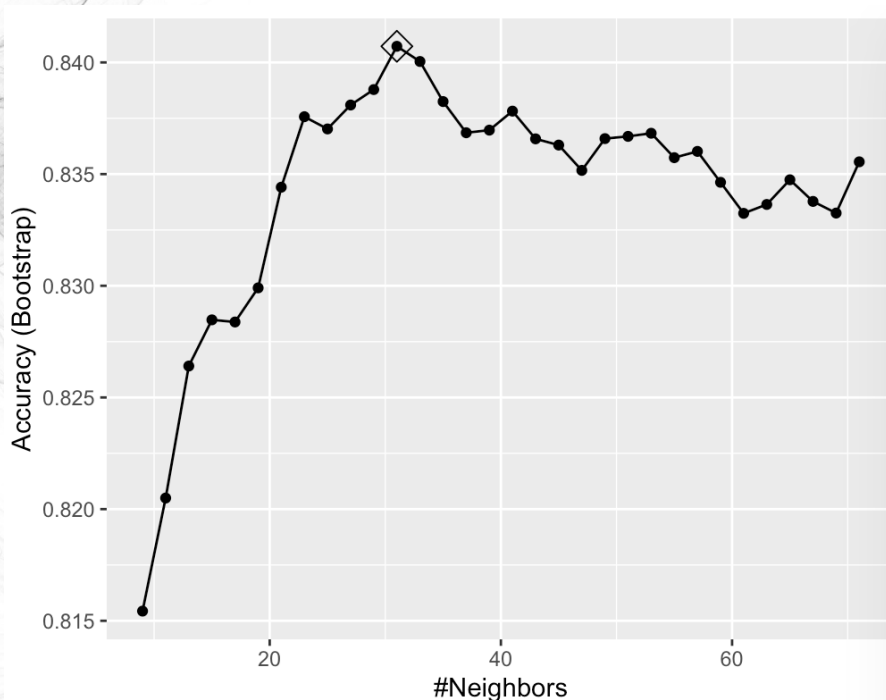

The caret package – parameter tuning



k in KNN is a parameter that refers to the number of nearest neighbors to include in the majority of the voting process.

[A Simple Introduction to K-Nearest Neighbors Algorithm | by Dhilip Subramanian | Towards Data Science](#)

For the kNN method in caret, the default is to try $k = 5, 7, 9$. We change this using the tuneGrid parameter. Here we show an example of trying out 30 k values between 9 and 67



```
R 4.2.2 · ~/ ➡  
> set.seed(42)  
> train_knn <- train(y ~ ., method = "knn",  
+                   data = mnist_27$train,  
+                   tuneGrid = data.frame(k = seq(9, 71, 2)))  
> ggplot(train_knn, highlight = TRUE)
```

The caret package – parameter tuning

```
R 4.2.2 · ~/ ↵  
> train_knn$bestTune  
      k  
12 31
```

Directly access the parameter that maximized the accuracy

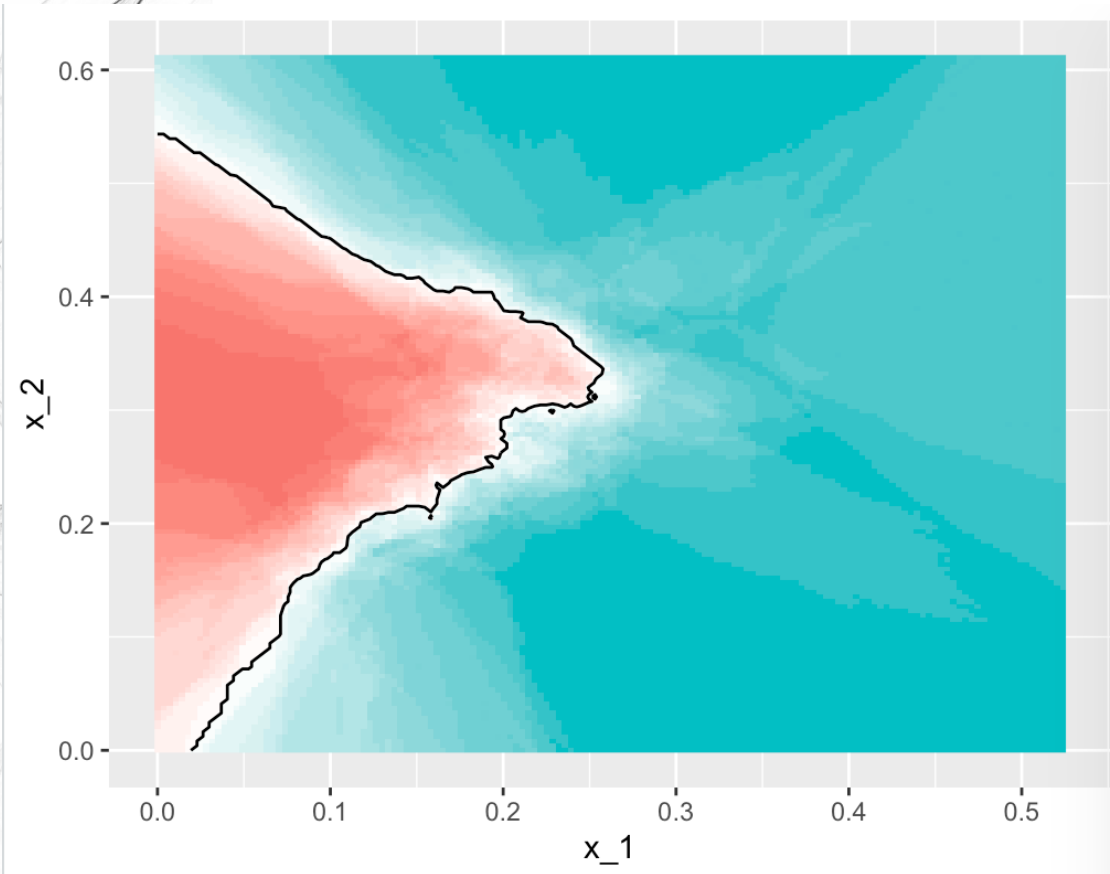
```
R 4.2.2 · ~/ ↵  
> train_knn$finalModel  
31-nearest neighbor model  
Training set outcome distribution:  
  
      2      7  
379 421
```

Directly access the best performing model

```
R 4.2.2 · ~/ ↵  
> confusionMatrix(predict(train_knn, mnist_27$test, type = "raw"),  
+                  mnist_27$test$y)$overall["Accuracy"]  
Accuracy  
0.845
```

The function predict will use this best performing model. Here is the accuracy of the best model when applied to the test set

The caret package



Compare the result of the final knn model to linear regression

```
R 4.2.2 · ~/ 
> plot_cond_prob <- function(p_hat=NULL){
+   tmp <- mnist_27$true_p
+   if(!is.null(p_hat)){
+     tmp <- mutate(tmp, p=p_hat)
+   }
+   tmp |> ggplot(aes(x_1, x_2, z=p, fill=p)) +
+     geom_raster(show.legend = FALSE) +
+     scale_fill_gradientn(colors=c("#F8766D", "white", "#00BFC4")) +
+     stat_contour(breaks=c(0.5), color="black")
+ }
> plot_cond_prob(predict(train_knn, mnist_27$true_p[,c(1,2)], type = "prob")[,2])
```

Work on all digits

```
R 4.2.2 · ~/ ↗  
> library(tidyverse)  
> library(dslabs)  
> mnist <- read_mnist()
```

Load the MNIST dataset from the dslabs package

```
R 4.2.2 · ~/ ↗  
> names(mnist)  
[1] "train" "test"
```

The dataset includes two components, a training set and test set

```
R 4.2.2 · ~/ ↗  
> dim(mnist$train$images)  
[1] 60000  784  
> dim(mnist$test$images)  
[1] 10000  784
```

Each of these components includes a matrix with features in the columns

```
R 4.2.2 · ~/ ↗  
> table(mnist$train$labels)
```

0	1	2	3	4	5	6	7	8	9
5923	6742	5958	6131	5842	5421	5918	6265	5851	5949

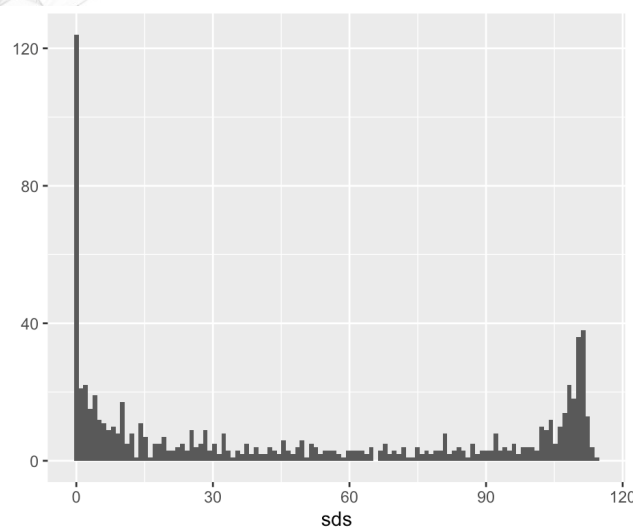
The labels vector shows the digit classes are about even

R 4.2.2 · ~/

```
> set.seed(42)
> index <- sample(nrow(mnist$train$images), 10000)
> x <- mnist$train$images[index,]
> y <- factor(mnist$train$labels[index])
>
> index <- sample(nrow(mnist$test$images), 1000)
> x_test <- mnist$test$images[index,]
> y_test <- factor(mnist$test$labels[index])
```

As we run on a laptop, and we want the training to finish in an hour, let's use a subset

In machine learning, we often transform features before running the machine algorithm. We can also remove features that are clearly not useful. We call these steps preprocessing. Examples of preprocessing include **standardizing the features, taking the log transform of some features, removing features that are highly correlated with others, and removing features with very few non-unique values or close to zero variation**

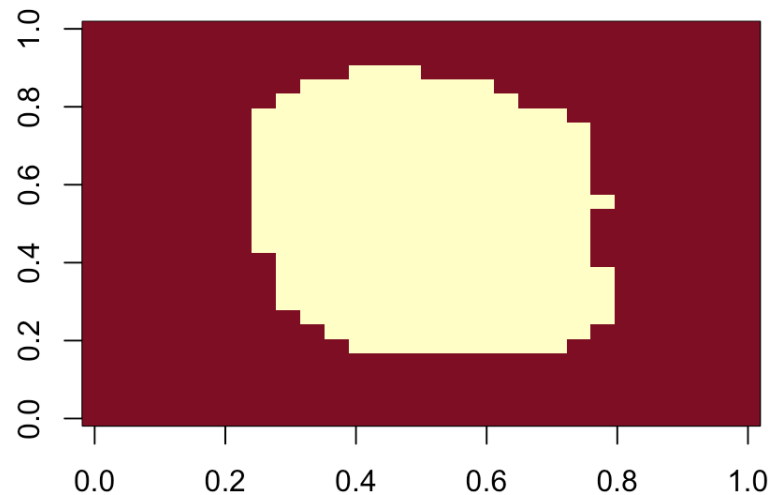


In our training data, there are some pixels that are always dark (contained no writing) or bright (contained a writing), let's consider removing these pixels

R 4.2.2 · ~/

```
> library(matrixStats)
> sds <- colSds(x)
> qplot(sds, bins=as.integer(max(sds)))
```

colSds: calculate column standard deviations



```
R 4.2.2 · ~/ ↗  
> col_index <- setdiff(1:ncol(x), nzv)  
> length(col_index)  
[1] 249
```

```
R 4.2.2 · ~/ ↗  
> colnames(x) <- 1:ncol(mnist$train$images)  
> colnames(x_test) <- colnames(x)
```

We can use the `nearZeroVar` function in `caret` to remove these pixels. We can see the columns recommended for removal using `image`

```
R 4.2.2 · ~/ ↗  
> nzv <- nearZeroVar(x)  
> image(matrix(1:784 %in% nzv, 28, 28))
```

So we end up keeping 249 features out of 784

Now we are ready to fit some models. Before we start, we need to add column names to the feature matrices as these are required by `caret`

R 4.2.2 · ~/

```
> train_knn <- train(x[,col_index], y,  
+                   method = "knn",  
+                   tuneGrid = data.frame(k = c(3,5,7)))
```

```
> train_knn
```

k-Nearest Neighbors

10000 samples

249 predictor

10 classes: '0', '1', '2', '3', '4', '5', '6', '7', '8', '9'

No pre-processing

Resampling: Bootstrapped (25 reps)

Summary of sample sizes: 10000, 10000, 10000, 10000, 10000, 10000, ...

Resampling results across tuning parameters:

k	Accuracy	Kappa
3	0.9307503	0.9229932
5	0.9315685	0.9239011
7	0.9313573	0.9236637

Accuracy was used to select the optimal model using the largest value.

The final value used for the model was k = 5.

R 4.2.2 · ~/

```
> control <- trainControl(method = "cv", number = 10, p = .9)  
train_knn_cv <- train(x[,col_index], y,  
+                   method = "knn",  
+                   tuneGrid = data.frame(k = c(3,5,7)),  
+                   trControl = control)  
  
train_knn_cv
```

Let's start with using kNN for model training. The default kNN used ~10 minutes on my Macbook M2. To try which is the best k , kNN in R by default does 25 bootstraps (more details in Chapter 29.5), each with 25% total training samples

A faster setup is to use a 10-fold cross validation for k selection

```
> n <- 1000
> b <- 2
> index <- sample(nrow(x), n)
> control_testRun <- trainControl(method = "cv", number = b, p = .9)
> train_knn_cv_testRun <- train(x[index, col_index], y[index],
+                               method = "knn",
+                               tuneGrid = data.frame(k = c(3,5,7)),
+                               trControl = control)
>
```

```
R 4.2.2 · ~/
> y_hat_knn <- predict(train_knn, x_test[, col_index])
> cm <- confusionMatrix(y_hat_knn, factor(y_test))
> cm$overall["Accuracy"]
Accuracy
0.954
```

```
R 4.2.2 · ~/
> cm$byClass[,1:2]
```

	Sensitivity	Specificity
Class: 0	0.9893617	0.9955850
Class: 1	1.0000000	0.9932356
Class: 2	0.9133858	0.9988545
Class: 3	0.9622642	0.9932886
Class: 4	0.9500000	0.9922222
Class: 5	0.9506173	0.9967356
Class: 6	0.9803922	0.9910913
Class: 7	0.9534884	0.9923414
Class: 8	0.9042553	0.9988962
Class: 9	0.9381443	0.9966777

In general, it is a good idea to try a test run with a subset of the data to get an idea of timing before we start running code that might take hours to complete. We can then increase n and b and try to establish a pattern of how they affect computing time to get an idea of how long the fitting process will take for larger values of n and b . You want to know if a function is going to take hours, or even days, before you run it.

Test out a trained model on test data



In this lecture

- Definition and notation
- Evaluations
- Decision boundary
- **What to use in practice**



What to use in practice?

- Supervised learning
 - Linear models: linear regression, logistic regression
 - **Lasso**
 - Ridge regression
 - Elastic net
 - Decision trees: **random forest** and gradient boosting (XGB)
 - **Support vector machines**
 - K-nearest neighbors
 - Neural networks



Lasso: Linear modeling with l_1 regularization

- Lasso's optimization goal:

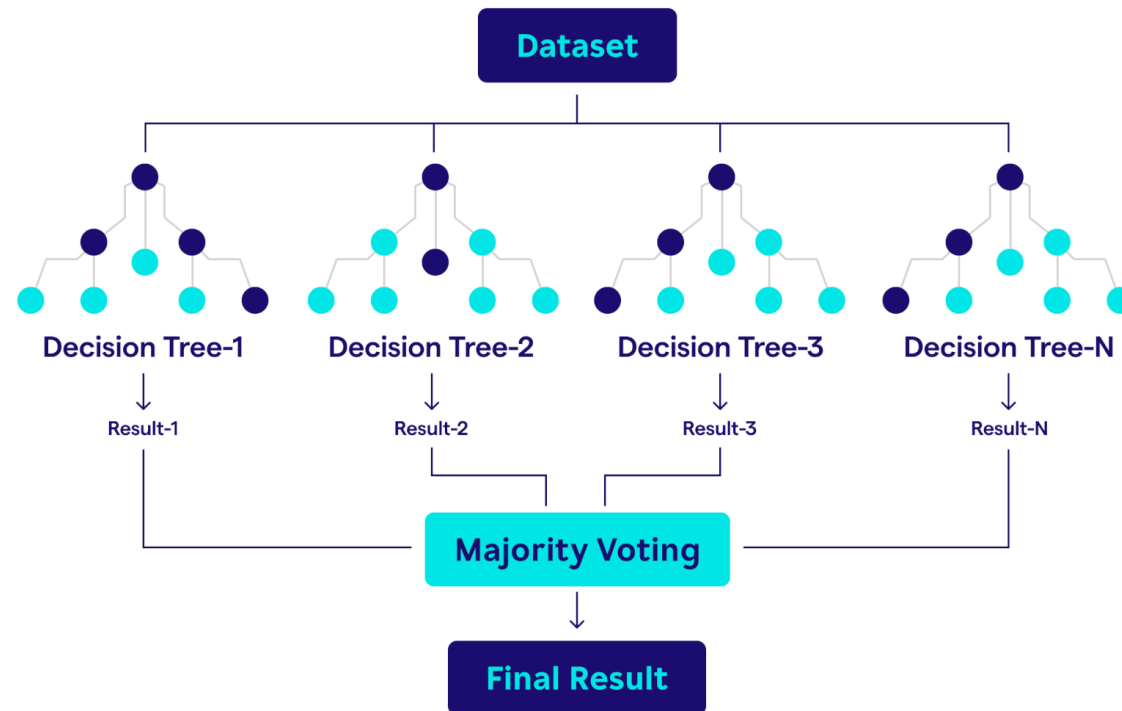
Least Absolute Shrinkage
and Selection Operator

$$\min_{\beta} \left\{ \sum_{i=1}^n \left(y_i - \sum_{j=1}^p \beta_j x_{ij} \right)^2 + \lambda \sum_{j=1}^p |\beta_j| \right\}$$

- n : number of observations in the dataset
- y_i : actual target value for observation i
- x_{ij} : value of feature j for observation i
- λ : regularization parameter
- p : number of features
- β_j : coefficient for the j -th feature (where $j = 1, 2, \dots, p$)
- $|\beta_j|$: absolute value of the j -th coefficient (ensures non-negative penalty)

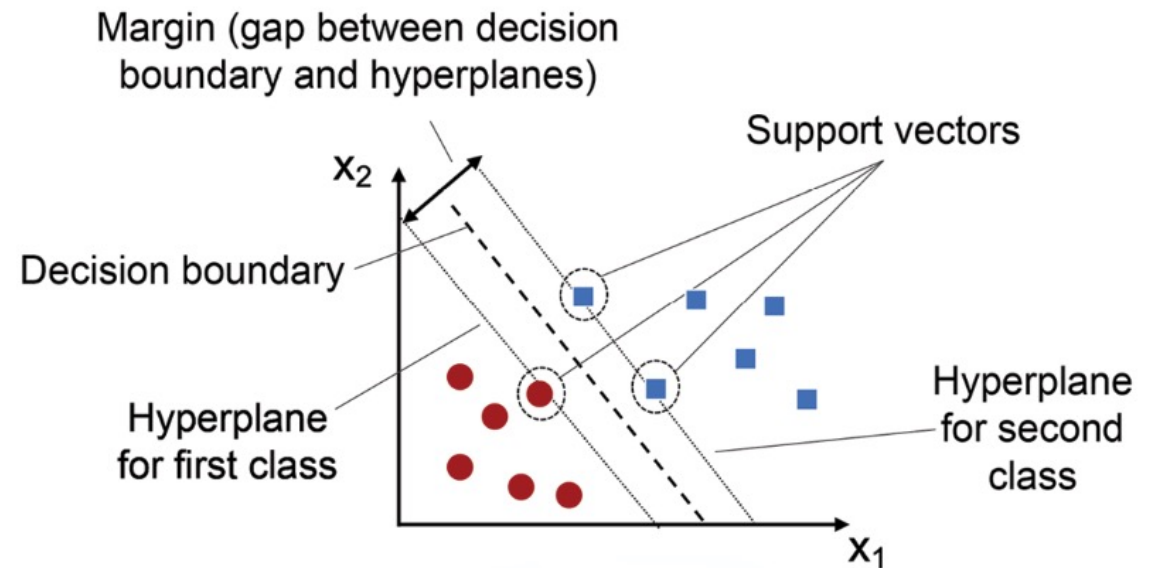
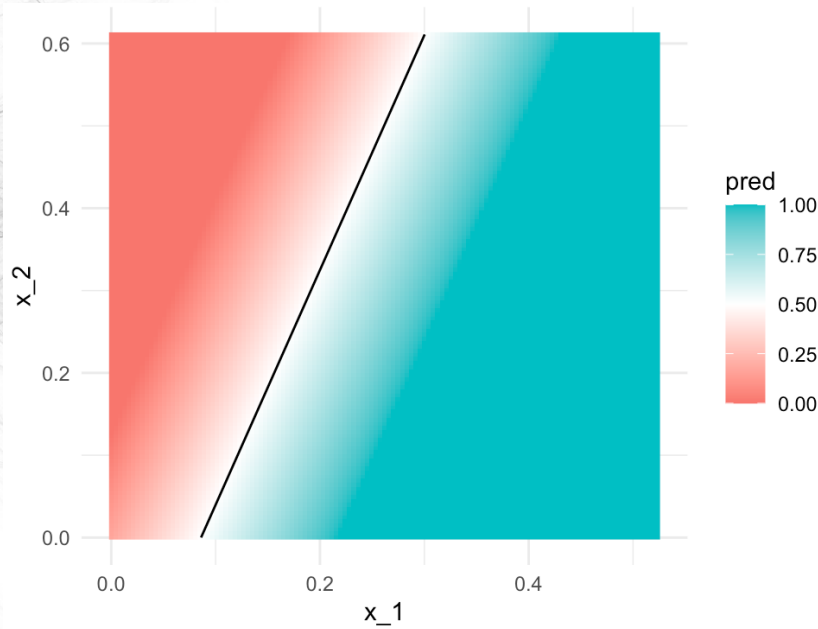
Random forest: decision tree with bootstrapping

- Each decision tree is constructed based on a subset of data (bootstrapping), and a subset of features.
- Each individual tree overfits, but it is reduced through ensembling many trees.



Support vector machines: expand the decision boundary

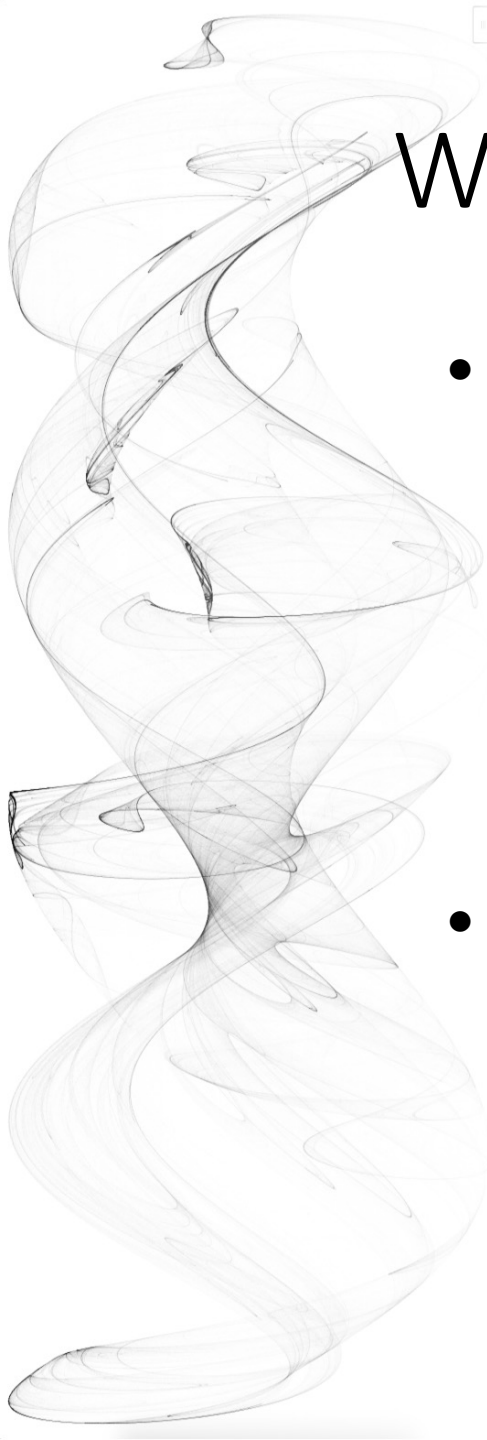
- Derive the decision boundary that has the largest “separation”/“margin”





Other types of learning

- Unsupervised learning: *e.g., clustering*
- Semi-supervised learning: *e.g., label propagation*
- Self-supervised learning: *e.g., BERT, GPT*
- Weakly supervised learning: *e.g., multiple instance learning*
- Meta-learning: *e.g., few-shot learning*
- Reinforcement learning: *e.g., AlphaGo*
- ...



What to use in practice?

- Python
 - scikit-learn: for evaluations and most machine learning algorithms
 - XGBoost: gradient boosting
 - PyTorch/tensorflow/keras: everything related to neural networks
 - transformers: modern transformer-based architectures
- R
 - caret: for most machine learning algorithms
 - torch for R, and other python packages via reticulate